



Lab Course Automatic Control II

OPT II – Model predictive control of a ball on plate system

Supervisor: Paulina Conrad

1 Introduction

In this practical experiment, a *ball on plate* system is considered. The moving ball on the plate represents a nonlinear dynamic system. The setup, which is shown in Figure 1, consists of an inner and an outer frame, whose axes of rotation are orthogonal to each other. This allows to move the ball to any point on the plate by deliberately tilting the frames. The rotation of the plates is generated by the actuation with two DC motors, which transmit the torque to the frames by means of a toothed belt drive. In order to determine the current position of the ball, a touch screen is used as the plate. It consists of two conductive layers separated by an insulating spacer. As soon as the ball is placed on the plate, contact occurs between the conductive layers and the position can be determined by applying reference voltages. Two additional sensors are located directly on the rotational axes of the two frames to determine the angles.

Within the scope of the experiment, a trajectory and path following controller based on *model predictive control* (MPC) should be developed, which has the goal to keep the ball on a given trajectory. In addition, the constraints of the system should be explicitly considered in the controller design. The designed model predictive controller will first be validated simulatively and then tested on the experimental setup, for which a real-time capable dSPACE system is used.

For the execution of this experiment, the contents of the lecture *Numerical Optimization and Model Predictive Control* are required. If necessary, re-read the most important topics with the help of [1].

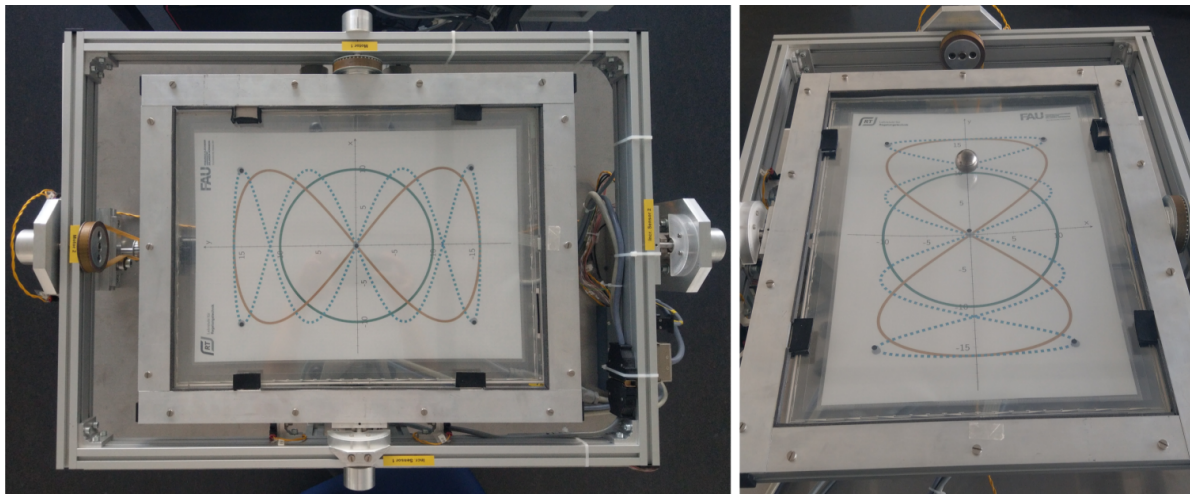


Figure 1: The ball on plate system available at the Chair of Automatic Control.

2 Experiment preparation

The basis for the design of a model predictive controller is the model of the system, which is derived in the first step.

For this purpose, the system is split into two separate *ball on stick* systems, which are subsequently considered independently of each other. The geometric relationships of a ball on stick system are shown in Figure 2. For modeling, it is assumed that the ball is constantly in contact with the stick and moves around its center of gravity only by rolling, i.e., the ball neither bounces nor slides. Choosing the stick angle θ and the ball position x_B as generalized coordinates of the system, the equations of motion can be derived using the *Lagrangian equations of second kind* to

$$0 = \left(\frac{J_B}{r_B^2} + m_B \right) \ddot{x}_B + m_B g \sin(\theta) - m_B x_B \dot{\theta}^2 \quad (1)$$

$$\tau = (m_B x_B^2 + J_S + J_B) \ddot{\theta} + 2m_B x_B \dot{x}_B \dot{\theta} + m_B g x_B \cos(\theta). \quad (2)$$

Here J_B denotes the moment of inertia of the ball, m_B its mass, r_B its radius, J_S the moment of inertia of the stick, g the gravitational acceleration and τ the torque applied to the stick. If the angular acceleration $\ddot{\theta}$ is now chosen as the new input u and by using the definition of

$$B = \frac{m_B}{\frac{J_B}{r_B^2} + m_B} \quad (3)$$

and the nonlinear invertible transformation

$$\tau = 2m_B x_B \dot{x}_B \dot{\theta} + m_B g x_B \cos(\theta) + (m_B x_B^2 + J_S + J_B) u \quad (4)$$

the system can be transferred into the state space representation

$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \\ \dot{x}_3 \\ \dot{x}_4 \end{bmatrix} = \begin{bmatrix} x_2 \\ B(x_1 x_4^2 - g \sin(x_3)) \\ x_4 \\ u \end{bmatrix} \quad (5)$$

with the state vector $\mathbf{x} = [x_B, \dot{x}_B, \theta, \dot{\theta}]^T$ (see [2]).

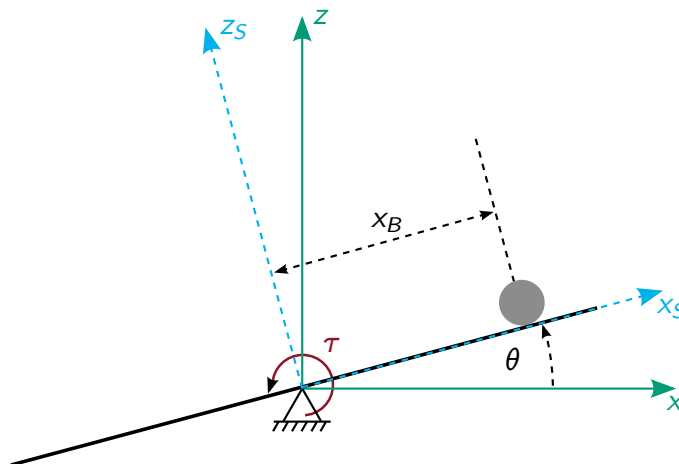


Figure 2: Geometric relations of a ball on stick system.

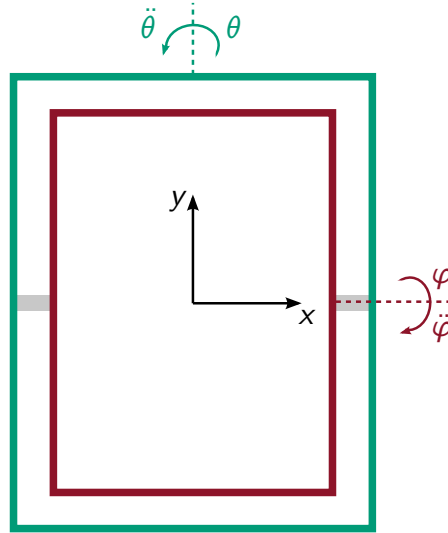


Figure 3: Definition of the coordinates on the ball on plate system.

The model of the complete ball on plate system is then obtained by interconnecting two independent ball on stick systems. The coordinate system definition used in this process is shown in Figure 3. The outer frame can be rotated about the y -axis, with a positive angle of rotation θ resulting in an acceleration of the ball in negative x -direction. Similarly, the inner plate, which is rotatably mounted in the outer, can be rotated about the x -axis. Thus with the state and control vectors

$$\mathbf{x} = [x_B, \dot{x}_B, \theta, \dot{\theta}, y_B, \dot{y}_B, \varphi, \dot{\varphi}]^T \quad (6)$$

$$\mathbf{u} = [\ddot{\theta}, \ddot{\varphi}]^T \quad (7)$$

the complete system dynamics results in

$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \\ \dot{x}_3 \\ \dot{x}_4 \\ \dot{x}_5 \\ \dot{x}_6 \\ \dot{x}_7 \\ \dot{x}_8 \end{bmatrix} = \begin{bmatrix} x_2 \\ B(x_1 x_4^2 - g \sin(x_3)) \\ x_4 \\ u_1 \\ x_6 \\ B(x_5 x_8^2 - g \sin(x_7)) \\ x_8 \\ u_2 \end{bmatrix}. \quad (8)$$

The ball has a mass of $m_B = 0.092$ kg, a radius $r_B = 14.3$ mm and its moment of inertia can be calculated by $J_B = \frac{2}{5} m_B r_B^2$.

In addition to the system dynamics (8), the physical constraints of the system should also be directly considered in the control. For the ball on plate system, these consist of the limits of the touch screen, resulting in constraints on the ball position in x and y directions

$$x_B \in [-0.1775 \text{ m}, 0.1775 \text{ m}] \quad (9)$$

$$y_B \in [-0.2365 \text{ m}, 0.2365 \text{ m}]. \quad (10)$$

Furthermore, the deflection of the two plates is limited to the maximum angles

$$\theta \in [-13^\circ, 13^\circ] \quad (11)$$

$$\varphi \in [-14^\circ, 14^\circ] \quad (12)$$

and due to the maximum available torque of the drive motors the input constraints results in

$$u_1 \in [-10 \text{ rad/s}^2, 10 \text{ rad/s}^2] \quad (13)$$

$$u_2 \in [-23 \text{ rad/s}^2, 23 \text{ rad/s}^2]. \quad (14)$$

In the first step, a position change of the ball from a starting point $\mathbf{p}_{B,0} = [x_{B,0}, y_{B,0}]^T$ to a target point $\mathbf{p}_{B,R}$ will be considered.

- (P1) *Formulate a suitable optimization problem to stabilize the ball at a given setpoint given the system dynamics (8) as well as the constraints (9) – (14). For this, transform the state-dependent constraints into standard form $\mathbf{h}(\mathbf{x}, \mathbf{u}) \leq \mathbf{0}$ and specify an appropriate cost functional to reach the control target.*

The numerical solution of the constrained, nonlinear optimization problem is realized with the MPC toolbox GRAMPC. The toolbox is based on an augmented Lagrangian method for the direct consideration of state-dependent constraints in combination with a underlying gradient method. The toolbox should already be familiar to you from the lab experiment *OPT I – Model predictive control of a laboratory crane*.

- (P2) *Realize the designed model predictive controller using the MPC toolbox GRAMPC. To do so, implement the system dynamics, the cost function and the constraints as well as the partial derivatives needed for the numerical solution in a `probfcn.c` file. Then create a corresponding initialization script `initData.m` and pass the weights of your cost function via the `userparam` vector.*

Note: Corresponding templates are available in GRAMPC under `examples/Template`. For the calculation of the partial derivatives you can use the Symbolic Math Toolbox from MATLAB.

- (P3) *Test the controller simulatively for different position changes of the ball, by creating the control loop via Simulink. Therefore use the Simulink template from GRAMPC in `examples/Template`. Use the dynamics (8) as the system model, as well as a prediction horizon $T_h = 1 \text{ s}$ and a sampling rate of $\Delta t = 1 \text{ ms}$.*

The actual goal of the lab experiment is to guide the ball along a certain trajectory. This task shall first be implemented in the form of a *trajectory tracking control*. Here, the target trajectory $\mathbf{p}_{B,R}(t)$, which explicitly depends on the time t , contains both the geometric trajectory and, via the time parameterization, the longitudinal ball dynamics, whereby a specific position *and* velocity of the ball is expected at any time. Thus, the trajectory tracking control corresponds to following a time-varying target position. The periodic nominal trajectories considered in this lab are a circle, a figure eight, and a double figure eight and can be explicitly described using sine and cosine by

$$x_{B,R}(t) = a_x \sin(b_x vt), \quad y_{B,R}(t) = a_y \cos(vt). \quad (15)$$

The parameters a_x and a_y indicate the amplitude in the respective direction, the parameter v defines the velocity and with the parameter b_x the type of the target trajectory can be selected (circle $b_x = 1$, figure eight $b_x = 2$, double figure eight $b_x = 4$).

- (P4) *Extend your problem to a trajectory tracking control. Pass the parameters of the target trajectory analogously to the weights via the `userparam` vector.*

Note: In GRAMPC, the time t that is available in the `probfcn.c` file in `lfct`, `Vfct` and the corresponding partial derivatives is the local MPC time running from 0 to T_h . This means you have to pass the current sample time t_k in each time step via the `userparam` vector accordingly to have the global time available in your problem formulation.

- (P5) Test your controller for the circle target trajectory ($b_x = 1$) with parameters $a_x = a_y = 0.1$ m and $v = 1.0$ rad/s. Adjust the weights in your cost function to achieve the best possible trajectory tracking.

Note: The goal of the experiment execution is to test your model predictive control developed in the preparation on the real system. Therefore, be sure to bring all relevant files (Simulink model, GRAMPC `probfcn`, initialization script, etc.) from the preparation to the execution of the experiment. Since MATLAB R2018b is installed on the lab computer, save your Simulink model in this version if you are using a newer MATLAB version to avoid complications (with File → Export Model to → Previous Model...).

3 Experiment execution

During the execution of the experiment, the controller designed and simulatively validated in the preparation is tested on the real system. Before starting the experiment, read the general instructions on carrying out the experiment in Section 3.1 carefully.

3.1 General instructions for experiment execution

- Make sure at all times that you keep sufficient distance to the moving parts of the experiment during execution, as large and abrupt deflections in the plate angles can occur in case of errors in the implementation / operation!
- If one of the frames reaches one of the end stops during the test, the system will automatically shut down. In this case, deactivate the control, remove the ball from the plate and bring the system back to horizontal position manually. Then press the reset button (grey box) of the experimental setup.
- Save at least one measurement for each task of the experimental setup. This is done by right-clicking in the plot window in ControlDesk and then clicking on "Save Displayed Data as New Measurement". Then the measurement data, which are now available at Project → Praktikum_RT2_MPC_BallAufPlatte → Measurements, can be saved as a MATLAB file and then loaded and visualized using MATLAB.
- Pay particular attention to the instructions in the single tasks during execution.

3.2 Execution

Copy your initialization script as well as your probfct.c file from the preparation to the C:\BallAufPlatte_MPC\code directory. Then rename the probfct file to "probfct_BallOnPlate.c" to ensure that the compiler will find it later. Next, open MATLAB on the lab computer. If a prompt about the dSPACE system appears, select "RTI1202". Confirm all further messages with OK. Then change to the C:\BallAufPlatte_MPC\code directory.

To make sure that your model predictive control designed in the preparation works in general, in a first step it should again be validated simulatively. To do this, open the Simulink model BallAufPlatte_MPC_Simulation.slx and the associated initialization script Init_Template_Simulation.m. After a simulation run, the movement of the ball will be animated and the relevant variables will be displayed.

(E1) *Integrate your model predictive controller implemented in the preparation into the Simulink template for controlling the ball on plate system and link the outputs with the calculated control inputs as well as the current target position of the ball. The current system states $\mathbf{x}_k$ and the current time t_k are available as inputs.*

Then test your controller for the same reference as in preparation task (P5).

After you have successfully validated the tracking control by simulation, open the Simulink model BallAufPlatte_MPC.slx. In this model, a measurement filtering and an inner velocity control for friction compensation have already been implemented.

- (E2) *Integrate your model predictive controller into the Simulink template for controlling the ball on plate system under Control group → MPC-based control. The inputs are the currently measured system states $\mathbf{x}_k$ and the current time t_k . Link the outputs with the outputs of your controller as well as the current target position of the ball.*

To ensure the real-time capability of the control on the dSPACE system, for all following tasks choose a prediction horizon (parameter Thor) $T_h = 1$ s, the MPC sampling time (parameter dt) of 1 ms, 5 gradient and 2 multiplier iterations (options MaxGradIter & MaxMultIter) and 20 grid points for numerical integration (option Nhor).

- (E3) *Adjust the options and parameters in your initialization script accordingly.*

To use the designed controller on the real system, the Simulink model must be compiled and loaded to the dSPACE system. This can be done either by the *Build-Model* icon in Simulink (rightmost in the top toolbar) or by the key combination *Ctrl-B*. Additionally, this will create a *variable description* file (.sdf) in the current directory that will be needed later. Make sure that the dSPACE Micro Lab Box is switched on, otherwise an error message will occur during compilation.

- (E4) *Execute your own initialization script as well as the already existing script Init_Template.m, which contains the parameters for the underlying control as well as the measurement filtering. Then compile the Simulink model and fix any error messages that may occur in the process.*

In the next step, start the program *dSPACE ControlDesk* and open the prepared project via File → Open → Open Project and Experiment → Root Directory → Symbol ‘new folder’ → select C:\BallAufPlatte_MPC\Praktikum_RT2_MPC_BallAufPlatte → Versuchsdurchfuehrung → OK.

The project provides a graphical user interface for controlling the ball on plate system and for displaying the target and current position of the ball, its velocity, and the control inputs. Use the Go Online and Start Measuring icons in the upper toolbar to start the online control and the visualization of the measured values. Before compiling a new version of your Simulink model, you must click Go Offline, otherwise an error message will appear during the building process. After **each** compile, you must load the new version of the .sdf file in ControlDesk using Project → Praktikum_RT2_MPC_BallAufPlatte\Hardware Configurations\Platform\ballaufplatte_mpc.sdf → right-click → Reload → Yes.

To enable efficient parameterization of your model predictive controller in online execution of the real setup, it makes sense to introduce the most important parameters (weights in the cost function and the parameters of the reference from (15)) as variable parameters that can be adjusted via the ControlDesk interface. This enables parameterization in online mode without having to recompile the Simulink model each time.

- (E5) *Define the weights of your cost function and the parameters of the target trajectories as MATLAB variables in an initialization script. Then insert the corresponding parameters into the userparam and xdes vectors in the Simulink model before passing them to the GRAMPC block.*

To be able to adjust the parameters in the ControlDesk interface, insert an input field for each variable via Layouting → Insert Instrument → Numeric Input. Then you can link the respective variable to the input field by right-click on the input field → Variables → Add.

Note: You must first recompile your Simulink model after the changes and reload the .sdf file, otherwise the parameters will not yet be available in ControlDesk.

Now your developed control system should be tested and parameterized on the real system.

- (E6) Switch on the control hardware of the ball on plate system (button on the back of the PC case) and push the reset button on the front of the PC case. If one of the two plates reaches its end stop during the test, the reset button must be pushed again. Then switch to online mode in the ControlDesk interface. Enable the motors for the two plates and switch to the “angle control” mode. This aligns the plates horizontally. In the mode “Move to initial position”, the ball is moved to the desired position.

Note: Always place the ball in “angle control” mode on the plate before switching to “Move to initial position” mode to ensure that a reasonable measurement of the ball’s position is available to the controller.

Bring the ball to the origin and then switch to “MPC-based control” mode and test your controller for stabilization of the origin first. To do this, select the parameters of the target trajectories to $a_x = a_y = 0$ and carefully steer the ball manually away from the origin. Adjust the weights to return the ball to the origin with minimal overshoot. Is the origin stabilized without steady-state error?

- (E7) Now test your controller for the developed trajectory tracking control. To do this, first use the circle as the target trajectory with the parameters

$$a_x = 0.1 \text{ m}, \quad a_y = 0.1 \text{ m}, \quad v = 1.0 \text{ rad/s}, \quad b_x = 1. \quad (16)$$

Adjust the weights in the cost function to produce the best possible trajectory tracking.

Note: Before switching to “MPC-based control” mode, bring the ball to the position $[x_{B,R}(0), y_{B,R}(0)]^T$ in “Move to initial position” mode.

- (E8) Manually disturb the system by slightly pushing the ball out of its path. Can the controller compensate for these manual disturbances? What behavior do you observe when you manually stop the ball at a point and release it after a certain time?

Afterwards, test the control for the simple figure eight as a reference trajectory with the parameters

$$a_x = 0.1 \text{ m}, \quad a_y = 0.16 \text{ m}, \quad v = 0.5 \text{ rad/s}, \quad b_x = 2 \quad (17)$$

and with the same weights as for the circle. Does this also yield a good control result for this reference trajectory?

An alternative to trajectory tracking control is the so-called *path following control*. The main difference is the decoupling of the velocity from the geometric path. This means that the target path $\mathbf{p}_{B,R}(s)$ no longer depends explicitly on the time t but on the position s along the path. In addition, a target velocity $v_R(s)$ must be specified, which is achieved by the controller analogously to the target position. The target paths can again be explicitly defined with the help of sine and cosine by

$$x_{B,R}(s) = a_x \sin(b_x s), \quad y_{B,R}(s) = a_y \cos(s), \quad (18)$$

where these are now no longer dependent on a target speed.

To determine the current position of the ball s along the target path, the minimization problem

$$s = \arg \min_s \|\mathbf{p}_B(t) - \mathbf{p}_{B,s}(s)\|_2 \quad (19)$$

must be solved continuously, which corresponds to an orthogonal projection of the current ball position $\mathbf{p}_B(t)$ onto the target path $\mathbf{p}_{B,R}(s)$. However, since the direct solution of (19) is very time consuming, the projection is directly integrated into the MPC optimization problem. For this purpose, the system dynamics (8) is extended by the auxiliary system

$$\dot{x}_9 = \dot{s} = u_3 \quad (20)$$

with the path velocity v_R as virtual input u_3 , which is determined as a free parameter by the optimization [6]. Since s is a virtual variable, the path position can be determined by integrating (20), which results in the current position for the next MPC step being $s_{k+1} = s(\Delta t)$.

- (E9) *Extend your problem formulation by the auxiliary system (20) and adapt your cost function to realize a path following control.*

Note: In addition to adjusting the target position in the cost function, note that you now also need an additional term that includes the deviation of the current ball velocity

$$v_B(t) = \sqrt{\dot{x}_B^2(t) + \dot{y}_B^2(t)} \quad (21)$$

from the target velocity v_R . In the following, the target velocity is assumed to be constant. Due to the root term (21), there is a singularity for $v_B = 0$ in the partial derivatives of the cost function according to the states (in dI/dx). Think about how to suppress this singularity.

- (E10) *First test the developed path following control **simulatively** in the existing Simulink template. Again, use a circle as target path with the parameters*

$$a_x = 0.1 \text{ m}, \quad a_y = 0.1 \text{ m}, \quad b_x = 1 \quad (22)$$

and a target velocity $v_R = 0.05 \text{ m/s}$ and adjust the weights for a good control result.

- (E11) *Now, test the developed path following control on the real system for the same target path as in task (E10).*

What differences do you recognize in the fundamental behavior if you choose the weights for the deviation from the target velocity large or small in comparison to the weights of the position error? Which fundamental difference to the trajectory following control becomes visible?

- (E12) *Again, manually disturb the system by “pushing” the ball slightly out of its path. Can the controller compensate for these manual disturbances? What behavior do you now observe when you manually stop the ball at a point and release it again after a certain time?*

Afterwards, test the control for the simple figure eight as target path with the parameters

$$a_x = 0.1 \text{ m}, \quad a_y = 0.16 \text{ m}, \quad b_x = 2 \quad (23)$$

and $v_R = 0.1 \text{ m/s}$ with the same weights as for the circle. Does this also yield a good control result?

References

- [1] K. Graichen. Numerical Optimization and Model Predictive Control. *Lecture script Friedrich-Alexander Universität Erlangen-Nürnberg*, 2023.
- [2] J. Hauser, S. Sastry, P. Kokotovic. Nonlinear control via approximate input-output linearization: The ball and beam example. *IEEE transactions on automatic control*, 37(3), 392-398, 1992.
- [3] T. Englert, A. Völz, F. Mesmer, S. Rhein, K. Graichen. GRAMPC documentation. sourceforge.net/projects/grampc, 2018.
- [4] B. Käpernick, K. Graichen. The gradient based nonlinear model predictive control software GRAMPC. In *Proc. European Control Conference (ECC)*, Seiten 1170-1175, Straßburg, Frankreich, 2014.
- [5] T. Englert, A. Völz, F. Mesmer, S. Rhein, K. Graichen. A software framework for embedded nonlinear model predictive control using a gradient-based augmented Lagrangian approach. *Optimization and Engineering*, doi.org/10.1007/s11081-018-9417-2, 2019.
- [6] T. Faulwasser, J. Matschek, P. Zometa, R. Findeisen. Predictive path-following control: Concept and implementation for an industrial robot. In *Proc. of the IEEE Int. Conf. on Control Applications*, 2013.